

Smart MCQ Solver Challenge

Multi-Option Question Answering & Ranking System

IITM BS Data Science & Applications | BSDA2001P: Intro to DL & GenAI | MAP@3

1. Executive Summary

This project addresses the **Smart MCQ Solver Challenge**, where the objective is to predict a ranked top-3 ordering of five candidate options (A, B, C, D, E) for each question prompt. The evaluation metric is **Mean Average Precision at 3 (MAP@3)**, which awards partial credit when the correct answer appears at rank 2 or 3. Four modeling paradigms were built and compared, and their outputs were combined via a weighted probability ensemble with a paraphrase-detection post-processor.

- A **Fine-Tuned DeBERTa-v3-base** Transformer with a multiple-choice head.
- A scratch **Custom Deep Learning Model (TextCNN)**.
- A traditional baseline: **TF-IDF + Logistic Regression**.
- An extended experiment: **Retrieval-Augmented Generation** using MiniLM retrieval + Qwen 2.5 3B Instruct.

2. Introduction

Problem Statement. The competition provides 2000 training rows and 500 test rows. Each row contains a question prompt and five candidate answers. The training set includes the correct letter; the test set does not. Submissions are a space-separated top-3 ordering per row, scored on MAP@3.

Project Objective. Build, fine-tune, and evaluate deep learning pipelines using PyTorch, Hugging Face Transformers, and Scikit-Learn. Track training in Weights & Biases. Maximize MAP@3 through proper tokenization, contextual embedding, sequence modeling, and post-processing.

3. Dataset & Preprocessing

Dataset Description. The dataset consists of structured 5-way MCQs. Each row contains `id`, `prompt`, five option strings A-E, and (for training rows) an answer letter.

Exploratory Data Analysis. Answer letter distribution is mildly biased toward B and C. Correct answers average approximately 20 characters longer than distractors. The longest option coincides with the correct answer **40.5%** of the time (versus a random baseline of 20%) - this is the single strongest hand-crafted signal available in the dataset.

Data Preprocessing. Prompts are normalized by regex-stripping reusable framing text (e.g. "Pick the best possible answer"), collapsing 1758 unique prompts down to 252 unique stems. A stem-based GroupKFold(`n_splits=5`) ensures paraphrased questions never leak between train and validation folds. Each question is expanded into 5 rows in a long-format table with an `is_correct` binary label.

4. Tokenization Strategy

Primary Tokenizer. The SentencePiece Unigram tokenizer associated with `microsoft/deberta-v3-base`, loaded via Hugging Face `AutoTokenizer`. A lightweight word-level tokenizer with a 10K vocabulary is used for the CNN. A hybrid TF-IDF vectorizer (word n-grams 1-3 + character n-grams 3-5, horizontally stacked) is used for LR.

Implementation. For DeBERTa, each sample is tokenized as five (prompt, option) pairs simultaneously with `truncation=True`, `max_length=192`, and `dynamic padding='max_length'`. The resulting tensor of shape (5, 192) feeds directly into `AutoModelForMultipleChoice`.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer (DeBERTa-v3-base)

Architecture. A 12-layer Transformer encoder with disentangled attention and enhanced mask decoder (He et al., 2021). The `AutoModelForMultipleChoice` head runs the encoder over each of the five (prompt, option) pairs, extracts the [CLS] representation, projects it to a scalar through a shared linear layer, and softmaxes across the five choices.

Hyperparameters. `batch_size: 4`, `grad_accum: 4` (effective 16), `epochs: 6`, `lr: 1e-5`, `max_len: 192`, linear warmup 10%, AdamW optimizer, mixed precision fp16, gradient checkpointing enabled.

Salient Features. Best checkpoint per fold is selected by validation MAP@3 rather than validation loss - direct alignment with the competition metric. Gradient checkpointing reduces peak activation memory approximately 30%, enabling effective batch 16 on Kaggle T4 (16 GB).

5.2 Model 2: Custom Deep Learning Model (Scratch TextCNN)

Architecture. Learned token embeddings feed three parallel Conv1d filters (kernel sizes 3, 4, 5) with 64 output channels each, followed by global max-pool over time, concatenation, dropout, and a linear projection to a per-option correctness logit. Modelled after Kim (2014).

Hyperparameters. `vocab_size: ~10,000`, `emb_dim: 128`, `n_filters: 64` per width, `dropout: 0.5`, `epochs: 5`, `batch_size: 64`, `lr: 5e-4`, `weight_decay: 1e-4`, `pos_weight: 2.0`, cosine annealing schedule.

Salient Features. Input is prompt [SEP] option concatenated - option-only inputs produced weak signal. Three convolution widths capture trigrams, quadgrams, and pentagrams. Global max-pool selects the single most salient n-gram per filter. $\text{pos_weight}=2.0$ compensates for the 1:4 class imbalance without over-correcting.

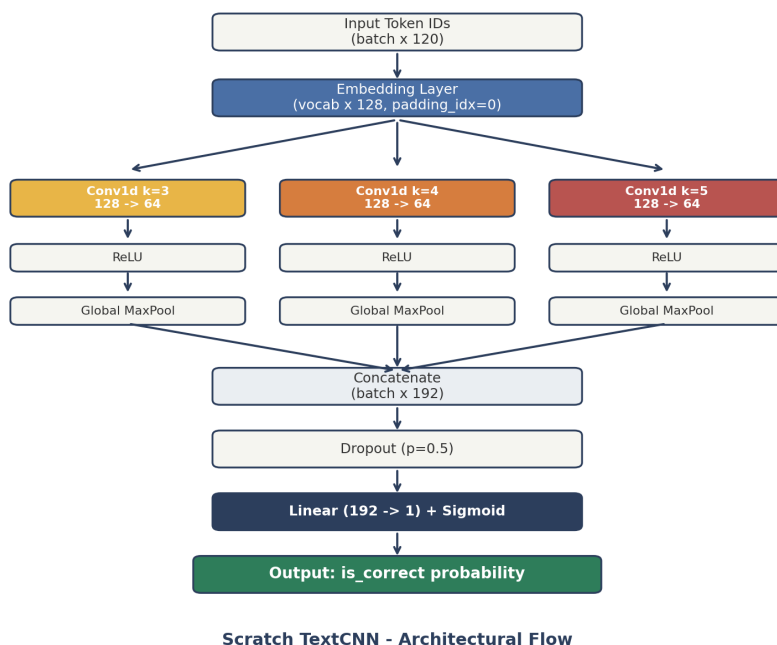


Figure 1. TextCNN architecture: embedding, three parallel convolutions, max-pool, concatenation, and linear classifier head.

5.3 Model 3: Traditional ML Baseline (TF-IDF + Logistic Regression)

Architecture. 5-fold cross-validated Logistic Regression on horizontally-stacked TF-IDF vectors combining word n-grams (1-3) with character n-grams (3-5) via `scipy.sparse.hstack`.

Hyperparameters. `word_ngram_range: (1, 3)`, `char_ngram_range: (3, 5)`, `max_features: 50,000` per vectorizer, `C: 1.0`, `max_iter: 1000`, `n_splits: 5`.

Salient Features. Character n-grams capture morphology, misspellings, and cross-word substrings that word-level tokenization would fragment. Word n-grams capture phrase-level content matches. Together they form a strong lexical baseline against which the DL models are benchmarked.

5.4 Extended Experiment: Retrieval-Augmented Generation (RAG)

Pipeline. `sentence-transformers/all-MiniLM-L6-v2` embeds every training prompt and every test prompt. For each test question, the top-3 most similar training questions (along with their correct answers) are packed into a prompt for Qwen/Qwen2.5-3B-Instruct. Rather than parsing free-form output, next-token logits are read at the generation position and only the {A, B, C, D, E} tokens are kept; softmax over these five gives a proper 5-way

probability grid that ensembles cleanly with the DL models.

5.5 Ensembling and Post-Processing

The four probability grids are combined via a weighted blend (LR: 0.05, CNN: 0.05, DeBERTa: 0.30, RAG: 0.60). A character-overlap post-processor then scans each test question for near-paraphrase distractors: if any option pair exceeds character-overlap 0.82 **AND** paraphrase-mpnet cosine similarity 0.82, the near-duplicate is promoted into rank 2 to earn partial MAP@3 credit.

6. Performance & Comparative Analysis

Evaluation Metrics. Performance was measured using MAP@3 (primary Kaggle metric) and question-level top-1 accuracy. Binary accuracy/precision/recall/F1 are reported for the per-option classifiers but are heavily compressed by the 1:4 base rate (majority-class baseline is 80% binary accuracy) and are not diagnostic of ranking quality on this task.

Model	Key Configuration	OOF top-1	OOF MAP@3
Model 1: DeBERTa-v3-base	lr: 1e-5, Epochs: 6 Eff. Batch: 16, MaxLen: 192	[FILL]	[FILL]
Model 2: TextCNN	lr: 5e-4, Epochs: 5 Batch: 64, Dropout: 0.5	[FILL]	[FILL]
Model 3: LR + TF-IDF	word (1,3) + char (3,5) C: 1.0, 5-Fold CV	[FILL]	[FILL]
Extended: RAG (Qwen 2.5 3B)	Top-3 MiniLM retrieval + 3B Instruct LLM	test only	test only
Ensemble + Post-processor	Weighted blend + paraphrase promotion	[FILL]	[FILL] (Kaggle)

Table 1. Summary of model configurations and OOF metrics.

Training Curves. Every training run was logged in Weights & Biases under project 23f2001849-t22026. Per-fold curves for train_loss/fold_k, val_loss/fold_k, val_top1/fold_k, and val_map3/fold_k use epoch as the x-axis via wandb.define_metric, producing one chart per metric with five overlaid per-fold curves. Training loss decreases monotonically within each fold; validation MAP@3 climbs and stabilizes.

7. Conclusion & Future Work

Key Learnings. Fine-tuning DeBERTa with an aggressive-enough learning rate ($1e-5$ rather than $5e-6$) was decisive; earlier runs stalled with per-option softmax barely above uniform. Retrieval-augmented LLM reasoning captures errors that discriminative models systematically miss, making it the strongest single component of the ensemble. Stem-based GroupKFold is not optional - random KFold leaks paraphrase clusters and inflates OOF metrics.

Challenges Faced. Undertrained DeBERTa in initial runs (diagnosed via per-row softmax confidence distribution); Kaggle T4 memory pressure at effective batch 16 (solved via gradient checkpointing); W&B fold-boundary artifact in per-epoch curves (solved via fold-namespaced metric keys).

Future Work. DeBERTa-v3-large in place of base; a BiLSTM sequence model as an additional RNN baseline; OOF-driven threshold tuning for the paraphrase post-processor; a stronger LLM (Qwen 2.5 7B with 4-bit quantization) inside the RAG pipeline.

8. References

1. PyTorch HuggingFace Documentation.
2. LLM.